

CS509 - PGSL

Assignment 4

Purpose. This document defines the algorithms, text-file input formats, expected outputs, and timing requirements for Assignment 4. The driver program must read the input file, prepare the required data structure, call the algorithm, and print both the final result and the algorithm execution time.

Timing rule. Measure only the algorithm execution time. File reading, input parsing, adjacency-list-to-CSR conversion, output printing, and other setup work must not be included.

1. Assignment Structure

Task Type	Algorithms	Work Mode
Individual task	Vertex Coloring (Graph Coloring) and PageRank; graph as input (CSR)	Individual
Buddy tasks	K-Means Clustering and FastMap	Pair of two students

- Programming language: C or C++. C++ is preferred.
- For graph algorithms, the text-file adjacency list must be converted to CSR before the algorithm is called.
- For the individual task, both Vertex Coloring and PageRank must be implemented. They operate on separate graph inputs as described in Sections 5 and 6, and each must be timed independently.
- Vertex Coloring uses an unweighted, undirected CSR graph. PageRank uses an unweighted, directed CSR graph.

2. Brief Explanation of the Algorithms

2.1 Vertex Coloring (Graph Coloring)

Graph (Vertex) Coloring assigns a color to every vertex of an undirected graph such that no two adjacent vertices share the same color, while using as few colors as possible. Finding the true minimum number of colors (the chromatic number) is NP-hard, so this assignment uses a greedy heuristic that produces a valid, though not necessarily minimum, coloring.

- Both correctness (i.e., no two adjacent vertices share a color) and the number of colors used must be reported.
- The same coloring routine must work for all required graph sizes; do not hard-code colors for specific test graphs.

2.1.1 Greedy Vertex Coloring (Welsh-Powell ordering) *Reference:* <https://graphstream-project.org/doc/Algorithms/Welsh-Powell>

- Compute the degree of every vertex from the CSR representation.
- Order vertices by non-increasing degree (Welsh-Powell heuristic) to reduce the number of colors used.

- Process vertices in that order. For each vertex, assign the smallest color index not currently used by any already-colored neighbour.
- Continue until every vertex has been assigned a color.

Real-Life Use Cases of Vertex Coloring:

- Register allocation in compilers (interference graphs).
- Timetabling and exam/course scheduling to avoid conflicts.
- Frequency/channel assignment in wireless networks.
- Map coloring and sudoku-style constraint problems.
- Computer vision: Graph-based image segmentation can represent pixels or regions as vertices. Graph coloring and related graph-labeling techniques can be used when neighboring regions need different labels.
- Graph Neural Networks (GNNs): Vertex coloring can be used as a graph-labeling problem and as a way to create node features or positional encodings that help distinguish structurally similar nodes.

2.2 PageRank *Reference: <https://web.stanford.edu/class/cs106m/meetings/03-pagerank>*

PageRank estimates the relative importance of each vertex in a directed graph based on the structure of incoming links, originally developed to rank web pages. A vertex's rank is high if it is linked to by many vertices, or by vertices that themselves have high rank.

$$PR(v) = (1 - d) / N + d * \sum_{u \rightarrow v} (PR(u) / \text{outdegree}(u))$$

- N is the total number of vertices and d is the damping factor (typically 0.85).
- All vertices are initialized with rank 1/N.
- A vertex with outdegree 0 (a dangling vertex) must not cause a divide-by-zero; distribute its rank contribution evenly across all vertices, or treat it as linking to all vertices.
- Iterate the update for all vertices simultaneously (using the previous iteration's values) until the total change (e.g., sum of absolute differences) is less than or equal to the tolerance, or the maximum number of iterations is reached.
- The sum of all PageRank values should remain approximately 1.0 after each iteration.

Real-Life Use Cases of PageRank:

- Web search engine ranking of pages.
- Influence and centrality ranking in social networks.
- Citation analysis and ranking of academic papers.
- Recommendation systems that propagate importance across a graph.
- Natural Language Processing (NLP): PageRank is used in extractive text summarization. Sentences are represented as graph vertices, with edges representing similarity between sentences. PageRank then identifies the most important sentences.
- Social network analysis: Users can be represented as vertices and relationships as edges. PageRank can identify influential or important users.
- Typical ML/AI use: For ranking/feature extraction.

2.3 K-Means Clustering (Reference: <https://cs229.stanford.edu/summer2023/cs229-notes7a.pdf>)

K-Means Clustering partitions N data points in D -dimensional space into K clusters, so that each point belongs to the cluster whose centroid (mean) is closest, minimizing the within-cluster sum of squared distances (WCSS).

- Initialize K centroids (the first K input points, in input order, are recommended for reproducibility).
- Assignment step: assign every point to the nearest centroid using Euclidean distance.
- Update step: recompute each centroid as the mean of the points currently assigned to it.
- Repeat the assignment and update steps until no point changes cluster (or the centroid shift is below the tolerance), or the maximum number of iterations is reached.
- If a cluster becomes empty during an update, keep its previous centroid unchanged for that iteration.

Real-Life Use Cases of K-Means Clustering:

- Customer and market segmentation.
- Image compression and color quantization.
- Document and text clustering.
- Anomaly/outlier detection and unsupervised pattern discovery.
- Most popular unsupervised ML algorithm: Healthcare and medical data analysis, Speech/audio processing, Feature engineering / preprocessing (Vector quantization, Data summarization, Model compression), Attention/token routing in efficient transformers,

2.4 FastMap (*Reference: <https://dl.acm.org/doi/10.1145/568271.223812>*)

FastMap is a heuristic algorithm that maps N objects, described only by a pairwise distance function, into a k -dimensional Euclidean space so that Euclidean distances in the new space approximate the original distances. It does this without ever forming the full distance matrix explicitly beyond what is provided as input, and without eigen-decomposition.

- For each of the k target dimensions, choose a pair of 'pivot' objects that are approximately the farthest apart, using a fast heuristic (pick a random object, find the object farthest from it, then find the object farthest from that one; repeat once or twice).
- Project every object onto the line joining the two pivots using the law of cosines to obtain its coordinate for the current dimension.
- Update ('deflate') the remaining pairwise distances to remove the contribution already captured by this dimension before computing the next dimension.
- Repeat until k coordinates have been produced for every object.

Real-Life Use Cases of FastMap:

- Dimensionality reduction for visualization of high-dimensional or non-vector data.
- Indexing and speeding up similarity search in multimedia databases.
- Preprocessing for clustering or nearest-neighbour search when only distances (not coordinates) are known.
- Bioinformatics: visualizing sequences or structures compared only via distance/similarity scores.
- Heuristic search / pathfinding in AI

3. General Input-File Rules

- Every test case must be stored in a separate .txt file.
- Use space-separated integer values unless otherwise stated. K-Means points, FastMap distances, and PageRank parameters may use floating-point values.
- Vertex numbering / object numbering must be consistent throughout the semester. The recommended convention is 0 to N-1.
- For graph inputs (Vertex Coloring, PageRank), the file must store the graph as an adjacency list.
- The driver reads the adjacency-list file and then calls the provided/helper conversion function to create CSR input before calling the graph algorithm.
- The CSR conversion time must not be included in the reported algorithm execution time.
- The required input sizes differ per algorithm because of practical time and memory limits; see Section 4.2 and Section 4.3 for the exact lists.
- You are expected to call the CSR conversion function from the previous assignment. DO NOT copy the code into this assignment.
- Vertex Coloring graphs are undirected and unweighted, so each edge must appear in the adjacency list of both endpoint vertices. Count each undirected edge once in E.
- PageRank graphs are directed. List only the outgoing edges of each vertex.
- Vertex Coloring test graphs may be connected or disconnected, but must contain no self-loops. PageRank test graphs should have every vertex reachable by at least one incoming or outgoing edge where practical; document any dangling vertices.

4. CSR Graph and Non-Graph Input Sizes

4.1 Graph Input Format and CSR Conversion

- The input format for Vertex Coloring and PageRank is an adjacency list.
- As you are aware, CSR (Compressed Sparse Row) is a compact format used to store sparse graphs efficiently. For a graph, the adjacency list is converted into arrays:
 - row_ptr: tells where each vertex's neighbour list starts and ends.
 - col_idx: stores the neighbouring vertex numbers.
 - values: not required for Vertex Coloring (unweighted graph); stores no data, or may be omitted. For PageRank, the graph is also unweighted; row_ptr and col_idx alone are sufficient.
- A helper function must be provided to convert the adjacency-list representation into CSR format before the algorithm runs. You are expected to call the CSR conversion function from the previous assignment. DO NOT copy the code into this assignment.
- The adjacency-list-to-CSR conversion is preprocessing. Its execution time must not be included in the reported algorithm runtime.
- For an algorithm that uses CSR, timing must begin only after the CSR representation has been prepared.
- K-Means Clustering and FastMap are exempt from CSR conversion because they read points or pairwise distances directly from their text files, not a sparse graph.

4.2 Required Graph Input Sizes (Vertex Coloring and PageRank)

Algorithm	Required Number of Vertices (V)	Note
Vertex Coloring	10, 100, 10,000, 50,000, 100,000	Keep large graphs sparse, approximately $E = 2V$ to $4V$ where practical.
PageRank	10, 100, 1,000, 10,000, 50,000	50,000 is the largest required size. 100,000 vertices is optional. Keep large graphs sparse and document E.

- The number of edges (E), graph type, and other relevant properties must also be recorded for each graph test case in the README.
- You can write your own random graph generator file, which takes the number of vertices and edges as input.
- In case on your system, for any input size, execution is not complete (likely core-dump, out-of-memory, excessive runtime, etc.), please mention that in your report.
- For the optional 100,000-vertex PageRank test, failure is acceptable if the required tests are completed; record the observed behavior instead of hiding it.

4.3 Required K-Means and FastMap Test Cases

K-Means Clustering and FastMap do not use graph size. To generate uniform report data while increasing the computation performed per run, use the required dataset sizes below.

Test	N (points/objects)	D (dimensions) / distance input	K / target dims k	Note
km_01	100	2	K = 3	Small, easy to visualize/verify by hand.
km_02	1,000	2	K = 5	Moderate size.
km_03	10,000	5	K = 8	Higher dimensionality.
km_04	100,000	5	K = 10	Largest required size; keep D moderate.
fm_01	10	full N x N distance matrix	k = 2	Verify projected distances against originals by hand.
fm_02	100	full N x N distance matrix	k = 2	Moderate size.
fm_03	1,000	full N x N distance matrix	k = 3	Larger object set.
fm_04	10,000	full N x N distance matrix	k = 3	Largest required size; optional 50,000 (documented if attempted).

- Required K-Means tests use K values of 3, 5, 8 and 10 across the four sizes above; the same implementation must work for any K, D and N supplied in the input file.
- Required FastMap tests use a target dimensionality k of 2 or 3; the same implementation must work for any N and k supplied in the input file.

- For the largest required K-Means and FastMap sizes, if execution does not complete on your system, document the observed behavior (core-dump, out-of-memory, excessive runtime) in the report instead of omitting the result.

5. Vertex Coloring Input and Output Test File Format

5.1 Unweighted Undirected Adjacency-List File Format

```
V E
u0 degree neighbor1 neighbor2 ...
u1 degree neighbor1 neighbor2 ...
...
u(V-1) degree neighbor1 neighbor2 ...
```

- V: number of vertices. E: number of undirected edges, counting each graph edge once even though it appears in both adjacency lists.
- Every edge must appear in the adjacency list of both endpoint vertices.
- No self-loops. Parallel edges between the same pair of vertices should not be listed more than once.
- For an isolated vertex, the general format is u 0; an isolated vertex is valid for Vertex Coloring and is simply assigned color 0.

5.2 Vertex Coloring Example Input

```
5 6
0 2 1 2
1 3 0 2 3
2 3 0 1 4
3 2 1 4
4 2 2 3
```

5.3 Expected Vertex Coloring Output

Print the color assigned to every vertex, the total number of colors used, and the execution time.

Algorithm: Greedy Vertex Coloring

Vertex colors:

0 0

1 1

2 2

3 0

4 1

Colors used: 3

Execution time: <value> ms

- The exact colors assigned may differ from the example depending on vertex ordering, provided the coloring is valid (no adjacent vertices share a color).
- The number of colors used may vary slightly between valid implementations; report the value your implementation produces.

6. PageRank Input and Output Test File Format

6.1 Directed Unweighted Adjacency-List File Format

```
V E
u0 outdegree neighbor1 neighbor2 ...
u1 outdegree neighbor1 neighbor2 ...
...
u(V-1) outdegree neighbor1 neighbor2 ...
DAMPING d
TOLERANCE epsilon
MAX_ITERATIONS n
```

- V: number of vertices. E: number of directed edges.
- List only the outgoing edges of each vertex.
- DAMPING is the damping factor d , $0 < d < 1$ (0.85 is recommended).
- Stop iterating when the total change in rank across all vertices is $\leq$ TOLERANCE, or when MAX_ITERATIONS has been reached.

6.2 PageRank Example Input

```
4 4
0 1 1
1 1 2
2 2 0 1
3 1 2
DAMPING 0.85
TOLERANCE 0.0001
MAX_ITERATIONS 100
```

6.3 Expected PageRank Output

Print the final PageRank value for every vertex, the number of iterations, convergence status, and the execution time.

```
Algorithm: PageRank
Damping: 0.85
Vertex ranks:
0 0.226283
1 0.267536
2 0.297557
3 0.208624
Sum of ranks: 1.000000
Iterations: <value>
Converged: true
Execution time: <value> ms
```

- If the tolerance is not reached before MAX_ITERATIONS, print Converged: false and still report the final ranks, iteration count and execution time.

7. K-Means Clustering Input and Output Test File Format

7.1 Points File Format

```
N D K
x0_1 x0_2 ... x0_D
x1_1 x1_2 ... x1_D
```

```
...
x(N-1)_1 x(N-1)_2 ... x(N-1)_D
MAX_ITERATIONS n
TOLERANCE epsilon
```

- N: number of points. D: number of dimensions per point. K: number of clusters to form.
- Each of the N following lines contains D space-separated floating-point coordinates.
- TOLERANCE applies to the maximum centroid shift between iterations (Euclidean distance); stop when it is $\leq$ TOLERANCE or MAX_ITERATIONS is reached.

7.2 K-Means Example Input

```
6 2 2
1.0 1.0
1.5 2.0
3.0 4.0
5.0 7.0
3.5 5.0
4.5 5.0
MAX_ITERATIONS 300
TOLERANCE 0.0001
```

7.3 Expected K-Means Output

Print the cluster assignment for every point, the final centroids, the number of iterations, the within-cluster sum of squares (WCSS), and the execution time.

```
Algorithm: K-Means Clustering
K: 2
Point assignments:
0 0
1 0
2 1
3 1
4 1
5 1
Final centroids:
0: 1.250000 1.500000
1: 4.000000 5.250000
WCSS: <value>
Iterations: <value>
Converged: true
Execution time: <value> ms
```

- For datasets with multiple valid clusterings (e.g., different local optima), the assignments may differ between correct implementations if a different centroid initialization is used; using the first K points as initial centroids, as recommended, keeps results reproducible.

8. FastMap Input and Output Test File Format

8.1 Pairwise Distance Matrix File Format

```
N K
d(0,0) d(0,1) ... d(0,N-1)
d(1,0) d(1,1) ... d(1,N-1)
```



```
...  
d(N-1,0) d(N-1,1) ... d(N-1,N-1)
```

- N: number of objects. K: target embedding dimensionality (typically 2 or 3).
- The distance matrix is symmetric with zero diagonal: $d(i,i) = 0$ and $d(i,j) = d(j,i)$.
- Distances are non-negative floating-point values and must satisfy the triangle inequality for a meaningful embedding.
- The full $N \times N$ matrix is provided for simplicity of parsing; students should still avoid unnecessary extra copies for large N.

8.2 FastMap Example Input

```
5 2  
0.0 3.0 4.0 5.0 6.0  
3.0 0.0 5.0 6.0 5.0  
4.0 5.0 0.0 3.0 4.0  
5.0 6.0 3.0 0.0 3.0  
6.0 5.0 4.0 3.0 0.0
```

8.3 Expected FastMap Output

Print the k-dimensional coordinates produced for every object, the pivot objects chosen for each dimension, and the execution time.

```
Algorithm: FastMap  
Target dimensions: 2  
Pivots per dimension:  
Dim 1: 1 3  
Dim 2: 0 4  
Object coordinates:  
0: 2.166667 1.802776  
1: 0.000000 3.400000  
2: 3.166667 0.500000  
3: 6.000000 1.100000  
4: 5.000000 0.000000  
Execution time: <value> ms
```

- Because pivot selection uses a randomized heuristic, exact coordinates may differ slightly between correct implementations. Report should note the pivot-selection strategy and, optionally, the average pairwise distance error between original and embedded distances.

9. Timing and Measurement Requirements

- Start the timer immediately before calling the algorithm.
- Stop the timer immediately after the algorithm finishes.
- Do not include file reading, input parsing, memory allocation done as setup, adjacency-list-to-CSR conversion, result printing, or file writing.
- Report time in milliseconds or seconds. Use the same unit consistently within a comparison table.
- For very fast inputs, repeated runs may be used and the average may be reported, provided the number of runs is documented.
- For every Vertex Coloring, PageRank, K-Means, and FastMap input file, report the algorithm time for that particular test case.

- For PageRank, all iterations of the rank-update loop are part of the timed algorithm section.
- For K-Means, both the assignment step and the centroid-update step, across all iterations, are part of the timed algorithm section.
- For FastMap, pivot selection, projection, and distance deflation for all k dimensions are part of the timed algorithm section.

10. Required Report File with Result Tables

10.1 Vertex Coloring Results Table

File	V	E	Colors Used	Valid?	Time	Status
color_10.txt	10	...	...	Yes/No	... ms	Pass/Fail
color_100.txt / color_10000.txt / color_50000.txt / color_100000.txt	100 / 10,000 / 50,000 / 100,000	...	...	...	...	...

10.2 PageRank Results Table

File	V	E	Dampin g	Top Vertex	Sum of Ranks	Iter. / Time	Status
pagerank_10.txt	10	...	0.85	...	~1.000	.../... ms	Pass/Fail
pagerank_100.txt - pagerank_50000.txt	100 / 1,000 / 10,000 / 50,000	...	0.85	...	~1.000	.../... ms	...

10.3 K-Means Results Table

File	N	D	K	Max Iter.	Actual Iter.	WCSS	Time	Status
km_01.txt	100	2	3	300	...	...	... ms	Pass/Fail
km_02.txt - km_04.txt	1,000 / 10,000 / 100,000	2 / 5	5 / 8 / 10	...	...	...	...	...

10.4 FastMap Results Table

File	N	Target k	Pivots (per dim)	Avg. Distance Error	Time	Status
fm_01.txt	10	2	...	...	... ms	Pass/Fail
fm_02.txt - fm_04.txt	100 / 1,000 / 10,000	2 / 3	...	...	...	...

11. Suggested Test-File Naming

Algorithm	Suggested Names
Vertex Coloring	color_10.txt, color_100.txt, color_10000.txt, color_50000.txt, color_100000.txt
PageRank	pagerank_10.txt, pagerank_100.txt, pagerank_1000.txt, pagerank_10000.txt, pagerank_50000.txt; optional: pagerank_100000.txt
K-Means Clustering	km_01.txt, km_02.txt, km_03.txt, km_04.txt
FastMap	fm_01.txt, fm_02.txt, fm_03.txt, fm_04.txt; optional: fm_05.txt (N = 50,000)

Upload a single report file with tables for each algorithm on Google Classroom.

12. Minimum Expected Driver Behaviour

- Accept the selected algorithm and input-file path through the common wrapper/menu or terminal arguments (built in Assignment 1).
- Read and validate the input file.
- For CSR-based tasks (Vertex Coloring, PageRank), construct the adjacency list and call the helper function to convert it to CSR.
- Call the selected algorithm.
- Print the final answer in a readable form.
- Print only the measured algorithm execution time as the reported timing.
- Return a clear error message for an invalid or missing input file.
- For invalid Vertex Coloring input, reject a self-loop, a mismatched degree/neighbour count, or an out-of-range vertex id.
- For invalid PageRank input, reject a non-positive damping factor, a damping factor ≥ 1 , a non-positive tolerance, or a non-positive MAX_ITERATIONS.
- For invalid K-Means input, reject a non-positive N, D, or K, a K greater than N, a non-positive tolerance, or a non-positive MAX_ITERATIONS.
- For invalid FastMap input, reject a non-square or non-symmetric distance matrix, a non-zero diagonal entry, a negative distance, or a target dimensionality k that is non-positive or $\geq N$.